동시성 및 스레드 안전성 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 동시성 처리와 스레드 안전성 메커니즘을 상세히 분석합니다. Rust의 강력한 타입 시스템과 소유권 모델을 활용하여 안전하고 효율적인 동시성을 구현하는 방법을 살펴봅니다.

동기화 프리미티브 사용 분석

1. RwLock (읽기-쓰기 락)

1.1 SessionDatabase 보호

```
pub struct Session {
pub(crate) db: RwLock<SessionDatabase>,
// ...
}
```

사용 패턴:

- **읽기 작업**: 토렌트 목록 조회, 상태 확인
- **쓰기 작업**: 토렌트 추가/제거, 상태 변경
- **성능 최적화**: 다중 읽기 동시 허용, 쓰기 시에만 배타적 접근

1.2 ManagedTorrent 상태 보호

```rust
pub  struct ManagedTorrent {
pub(crate) locked: RwLock<ManagedTorrentLocked>,
// ...
}

pub(crate) struct ManagedTorrentLocked {
pub(crate) paused: bool,
pub(crate) state: ManagedTorrentState,
pub(crate) only_files: Option<Vec<usize>>,
}
```

보호 대상:

- 토렌트 상태 (Initializing, Paused, Live, Error)
- 일시정지 플래그
- 선택된 파일 목록

1.3 TorrentStateLocked (Live 상태)

```rust
pub(crate) struct TorrentStateLocked {
pub(crate) chunks: Option<ChunkTracker>,
file_priorities: FilePriorities,
inflight_pieces: HashMap<ValidPieceIndex, InflightPiece>,
fatal_errors_tx: Option<tokio::sync::oneshot::Sender<anyhow::Error>>,
unflushed_bitv_bytes: u64,
```

```
}
```

보호 대상:

- 청크 추적기 (다운로드 진행 상황)
- 파일 우선순위
- 진행 중인 조각 요청
- 오류 채널

2. Arc (원자적 참조 카운팅)

2.1 Session 공유

```
pub fn new(default_output_folder: PathBuf) -> BoxFuture<'static, anyhow::Result<
```

공유 패턴:

- 모든 토렌트가 Session 참조 보유
- HTTP API가 Session 참조 보유
- 백그라운드 태스크들이 Session 참조 보유

2.2 ManagedTorrentShared 구조

```
pub struct ManagedTorrentShared {
pub id: TorrentId,
pub info_hash: Id20,
pub(crate) spawner: BlockingSpawner,
pub trackers: HashSet<url::Url>,
```

```
pub peer_id: Id20,
pub span: tracing::Span,
pub(crate) options: ManagedTorrentOptions,
pub(crate) connector: Arc<StreamConnector>,
pub(crate) storage_factory: BoxStorageFactory,
pub(crate) session: Weak<Session>,
pub(crate) magnet_name: Option<String>,
}
```

설계 원칙:

- 불변 데이터만 공유
- 순환 참조 방지 (Weak 참조 사용)
- 생명주기 관리 자동화

2.3 약한 참조 (Weak) 사용

```
pub(crate) session: Weak<Session>,
```

목적:

- 순환 참조 방지
- 메모리 누수 방지
- 우아한 종료 지원

3. AtomicUsize 및 원자적 연산

3.1 토렌트 ID 생성

```
pub struct Session {
next_id: AtomicUsize,
// ...
}


// 사용 예시
self.next_id.fetch_add(1, std::sync::atomic::Ordering::Relaxed)
```

특징:

- Lock-free 카운터
- 고성능 ID 생성
- 메모리 순서 최적화 (Relaxed)

3.2 통계 카운터

```
macro_rules! gen_stats {
($atomic_name:ident $snapshot_name:ident, [$($stat_name:ident $stat_ty:tt),*], .
pub struct $atomic_name {
$(
pub $stat_name: stype!(atomic $stat_ty),
)*
}
impl $atomic_name {
pub fn $stat_name(&self, value: $stat_ty) {
```

```
self.$stat_name.fetch_add(value, std::sync::atomic::Ordering::Relaxed);
}
}
};
}
```

생성된 구조체들:

- `ConnectStatsAtomic` : 연결 통계
- `PeerCountersAtomic` : 피어 통계
- `AtomicStats` : 토렌트 통계

4. DashMap (동시성 해시맵)

4.1 피어 상태 관리

```
pub(crate) struct PeerStates {
pub states: DashMap<PeerHandle, Peer>,
// ...
}
```

특징:

- Lock-free 해시맵
- 높은 동시성 성능
- 세밀한 락킹 (샤딩)

사용 패턴:

```
// 피어 추가
match self.states.entry(addr) {
Entry::Occupied(_) => None,
Entry::Vacant(vac) => {
vac.insert(Peer::new_with_outgoing_address(addr));
Some(addr)
}
}

// 피어 접근
self.states.get(&addr).map(|e| f(e.value()))
```

비동기 태스크 스케줄링

1. spawn_with_cancel 패턴

1.1 기본 구조

```
pub fn spawn(
&self,
span: tracing::Span,
name: &'static str,
fut: impl std::future::Future<Output = anyhow::Result<()>> + Send + 'static,
) {
```

```
spawn_with_cancel(span, name, self.cancellation_token.clone(), fut);
}
```

특징:

- 구조화된 동시성
- 자동 정리 보장
- 추적 가능한 태스크

1.2 계층적 취소 토큰

```
pub struct Session {
cancellation_token: CancellationToken,
_cancellation_token_drop_guard: DropGuard,
// ...
}
```

계층 구조:

```
Session CancellationToken
├── DHT Task
├── Tracker Task
├── Torrent 1
| ├── Peer Manager
| ├── Piece Requester
| └── File Writer
```

```
└── Torrent 2
└── ...
```

2. 백그라운드 태스크 관리

2.1 리스너 태스크

```
if let Some(tcp) = listen.tcp_socket.take() {
session.spawn(
debug_span!(parent: session.rs(), "tcp_listen", addr = ?listen.addr),
"tcp_listen",
{
let this = session.clone();
async move { this.task_listener(tcp).await }
},
);
}
```

2.2 디스크 쓰기 워커

```
session.spawn(
debug_span!(parent: session.rs(), "disk_writer"),
"disk_writer",
async move {
```

```
while let Some(work) = disk_write_rx.recv().await {
spawner.spawn_block_in_place(work);
}
Ok(())
},
);
```

3. BlockingSpawner 최적화

3.1 런타임 감지

```
impl Default for BlockingSpawner {
fn default() -> Self {
let allow_block_in_place = match tokio::runtime::Handle::current().runtime_flavo
tokio::runtime::RuntimeFlavor::CurrentThread => false,
tokio::runtime::RuntimeFlavor::MultiThread => true,
_ => true,
};
Self::new(allow_block_in_place)
}
}
```

3.2 블로킹 작업 처리

```
pub fn spawn_block_in_place<F: FnOnce() -> R, R>(&self, f: F) -> R {
if self.allow_tokio_block_in_place {
return tokio::task::block_in_place(f);
}
f()
}
```

최적화 전략:

- 멀티스레드 런타임: `block_in_place` 사용
- 싱글스레드 런타임: 직접 실행
- CPU 집약적 작업 분리

취소 메커니즘

1. CancellationToken 활용

1.1 토큰 생성 및 전파

```
let token = opts.cancellation_token.take().unwrap_or_default();
let child_token = token.child_token();
```

1.2 취소 감지

```
tokio::select! {
result = main_task => result,
```

```
_ = cancellation_token.cancelled() => {
// 정리 작업
Ok(())
}
}
```

2. 우아한 종료

2.1 DropGuard 사용

```
pub struct Session {
_cancellation_token_drop_guard: DropGuard,
// ...
}
```

동작 방식:

- Session 드롭 시 자동 취소
- 모든 자식 태스크 정리
- 리소스 누수 방지

2.2 상태 정리

```
fn stop_with_error(&self, error: anyhow::Error) {
let mut g = self.locked.write();
match g.state.take() {
ManagedTorrentState::Live(live) => {
```

```
if let Err(err) = live.pause() {
warn!("error pausing live torrent during fatal error handling: {err:#}");
}
}
// ... 기타 상태 처리
};
g.state = ManagedTorrentState::Error(error)
}
```

데드락 방지 전략

1. 락 순서 규칙

1.1 문서화된 규칙

```
// NOTE: deadlock notice:
// peers and stateLocked are behind 2 different locks.
// if you lock them in different order, this may deadlock.
//
// so don't lock them both at the same time at all, or at the worst lock them in
// same order (peers one first, then the global one).
```

1.2 락 계층 구조

```
1. PeerStates (DashMap)

2. TorrentStateLocked (RwLock)

3. SessionDatabase (RwLock)
```

2. 락 보유 시간 최소화

2.1 TimedExistence 패턴

```rust
pub struct TimedExistence<T> {

value: T,

reason: &'static str,

start: Instant,

}


impl<T> Drop for TimedExistence<T> {

fn drop(&mut self) {

let elapsed = self.start.elapsed();

if elapsed > Duration::from_millis(10) {

warn!(reason = self.reason, elapsed_ms = elapsed.as_millis(), "slow lock");

}

}

}
```

목적:

- 락 보유 시간 모니터링
- 성능 병목 지점 식별
- 데드락 위험 감소

3. Lock-free 데이터 구조 활용

3.1 원자적 카운터

```
// 락 없는 통계 업데이트
atomic_inc(&self.stats.queued);
atomic_inc(&self.session_stats.queued);
```

3.2 DashMap 활용

```
// 세밀한 락킹으로 경합 감소
self.states.get_mut(&addr)
```

성능 최적화 기법

1. 메모리 순서 최적화

1.1 Relaxed 순서 사용

```
self.next_id.fetch_add(1, std::sync::atomic::Ordering::Relaxed)
self.checked_bytes.load(std::sync::atomic::Ordering::Relaxed)
```

이유:

- 순서 보장 불필요한 경우
- 성능 향상
- 캐시 일관성 오버헤드 감소

2. 읽기 최적화

2.1 RwLock 읽기 편향

```
// 빈번한 읽기 작업
let g = self.locked.read();
match &g.state {
// 상태 확인만 수행
}
```

2.2 Arc 복제 최소화

```
// 필요한 경우에만 복제
let session = self.session.upgrade()?;
```

3. 동시성 제한

3.1 세마포어 활용

```
pub(crate) concurrent_initialize_semaphore: Arc<tokio::sync::Semaphore>,
```

```
// 사용
```

```
let _permit = concurrent_init_semaphore.acquire().await?;
```

목적:

- 리소스 사용량 제어
- 시스템 과부하 방지
- 공정한 리소스 분배

3.2 피어 연결 제한

```
peer_semaphore: Arc<Semaphore>,
```

오류 처리 및 복구

1. 오류 전파

1.1 채널 기반 오류 처리

```
fatal_errors_tx: Option<tokio::sync::oneshot::Sender<anyhow::Error>>,
```

```
// 오류 발생 시
if let Some(tx) = self.fatal_errors_tx.take() {
let _ = tx.send(error);
}
```

2. 부분 실패 처리

2.1 피어별 독립적 처리

```
// 한 피어 실패가 다른 피어에 영향 없음
self.drop_peer(handle);
```

2.2 상태 복구

```
// Error 상태에서 Initializing으로 복구
Error => {
let initializing = Arc::new(TorrentStateInitializing::new(/* ... */));
g.state = ManagedTorrentState::Initializing(initializing);
}
```

모니터링 및 디버깅

1. 추적 가능한 태스크

1.1 구조화된 로깅

```
spawn_with_cancel(
debug_span!(parent: span.clone(), "initialize_and_start"),
"initialize_and_start",
token.clone(),
async move { /* ... */ }
);
```

2. 성능 메트릭

2.1 원자적 통계

```
pub struct AtomicStats {
pub downloaded_bytes: AtomicU64,
pub uploaded_bytes: AtomicU64,
pub download_speed_bps: AtomicU64,
pub upload_speed_bps: AtomicU64,
}
```

3. 데드락 감지

3.1 타임아웃 기반 감지

```
if elapsed > Duration::from_millis(10) {
warn!(reason = self.reason, elapsed_ms = elapsed.as_millis(), "slow lock");
}
```

결론

rqbit의 동시성 및 스레드 안전성 설계는 다음과 같은 특징을 가집니다:

1. **계층적 동기화**: RwLock, Arc, Atomic의 적절한 조합
2. **구조화된 동시성**: 취소 토큰을 통한 생명주기 관리
3. **성능 최적화**: Lock-free 구조와 메모리 순서 최적화

4. **데드락 방지**: 명확한 락 순서와 보유 시간 최소화

5. **확장 가능한 설계**: DashMap과 세마포어를 통한 동시성 제어

6. **강력한 오류 처리**: 부분 실패와 복구 메커니즘

7. **모니터링 지원**: 성능 메트릭과 디버깅 도구

이러한 설계를 통해 높은 성능과 안정성을 동시에 달성하는 동시성 시스템을 구현했습니다.